HCMC University Of Technology
Faculty of Computer Science & Engineering

Course: Operating Systems
Assignment - Simple Operating System

October 17, 2026

Goals: The objective of this assignment is to simulate major components of a simple operating system, such as the scheduler, synchronization mechanisms, and physical and virtual memory operations.

Contents: In detail, students will practice with three major modules: the scheduler, synchronization, and the mechanism for memory allocation from virtual to physical memory.

- scheduler
- synchronization
- memory-allocation operations from virtual to physical memory

In addition, students will practice designing and implementing a simple operating system programming interface through system calls.

Results: After this assignment, students will partially understand the principles of a simple OS. They will be able to explain and illustrate the roles of key OS modules.

Contents

1 Introduction **3**
1.1 An overview . . . 3
1.2 Source Code . . . 4
1.3 Processes . . . 5
1.4 How to Create a Process? . . . 7
1.5 How to Run the Simulation . . . 7
1.6 How to Write the Kernel Interface . . . 8
1.6.1 The kernel structure . . . 8
1.6.2 The system call . . . 8
1.6.3 Adding a system call to Simple Operating System . . . 9

2 Implementation **12**
2.1 Scheduler . . . 12
2.2 Memory Management . . . 13
2.2.1 The process memory layout . . . 13
2.2.2 The virtual memory mapping in each process . . . 15
2.2.3 The system's physical memory . . . 17
2.2.4 Paging-based address translation scheme . . . 18
2.2.5 Wrapping-up all paging-oriented implementations . . . 19
2.3 Co-processors . . . 21
2.3.1 Asynchronous operation and bus address . . . 21
2.3.2 Direct Memory Access - DMA . . . 22
2.3.3 Graphic Processor Unit - GPU . . . 23
2.3.4 I/O acceleration technologies . . . 24
2.4 Put It All Together . . . 24

3 Submission **26**
3.1 Source code . . . 26
3.2 Requirements . . . 26
3.3 Report . . . 26
3.4 Grading . . . 27
3.5 Code of ethics . . . 27

1 Introduction

1.1 An overview

This assignment simulates a simple operating system to help students understand the fundamental concepts of scheduling, synchronization, and memory management. Figure 1 shows the overall architecture of the *operating system* we are going to implement. In general, the OS manages two *virtual* resources, CPU(s)/co-processor(s) and RAM, by using two core components:

i. Scheduler and dispatcher determines which process is allowed to run on which CPU.

ii. Virtual memory isolates the memory space of each process from others. Physical RAM is shared by multiple processes, but each process is unaware of the others. This is done by giving each process its own virtual memory space, and the virtual memory engine maps and translates the virtual addresses provided by processes to the corresponding physical addresses.

iii. Co-processor is a secondary hardware processor designed to supplement the functions of the primary processor (the CPU). By offloading tasks to dedicated hardware, a co-processor frees up the CPU to handle general system management and significantly boosts overall system performance.

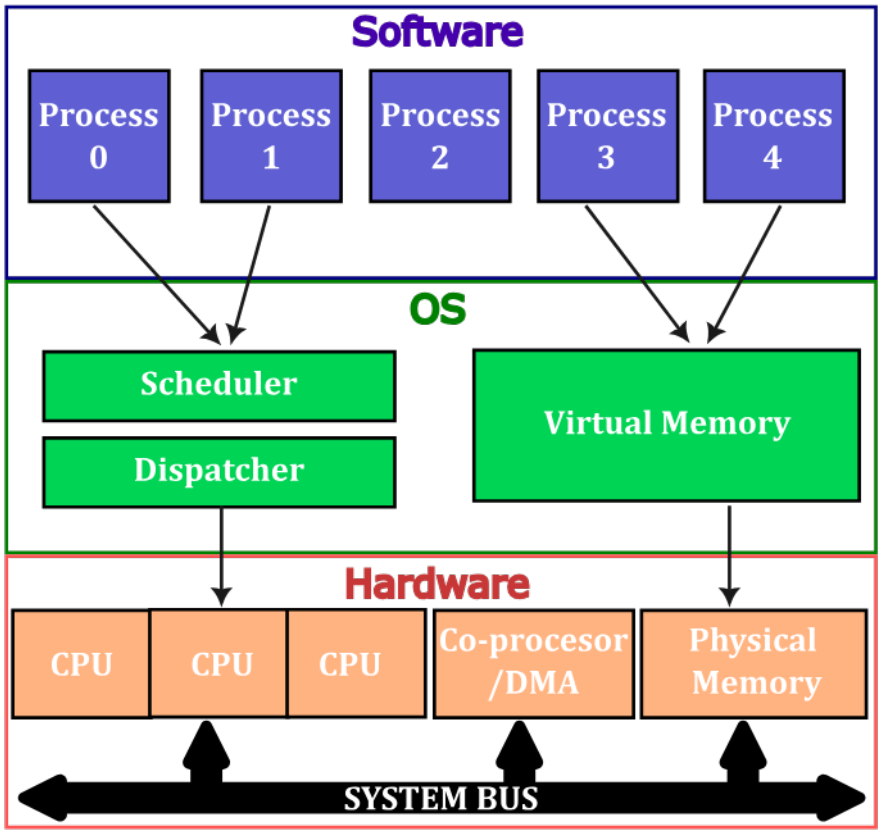


Figure 1: The general view of key modules in this assignment

For example, a direct memory access (DMA) engine transfers data between memory and devices with minimal CPU involvement; graphics processors accelerate graphics and vector or matrix computations; and network adapters can accelerate TCP operations such as checksums and segmentation. Offloading these tasks lets the CPU focus on user-oriented works.

The design supports dual-mode operation, which ensures system protection and controlled access to hardware resources. The OS operates in two modes: user mode and kernel mode, distinguished by a hardware-supported mode bit that indicates whether the CPU is executing user code or privileged kernel code. This mechanism guarantees that user programs cannot execute privileged instructions to preserve system stability and security.

Through these modules, the OS allows multiple user-created processes to share and use *virtual* computing resources. Therefore, in this assignment, we focus on implementing the scheduler/dispatcher and the virtual memory engine.

1.2 Source Code

After downloading the source code of the assignment in the *Resource* section on the portal platform and extracting it, you will see the source code organized as follows.

- Header files
  - `timer.h`: defines the timer for the whole system.
  - `cpu.h`: defines functions used to implement the virtual CPU.
  - `queue.h`: defines functions used to implement the queue that holds process PCBs.
  - `sched.h`: defines functions used by the scheduler.
  - `mem.h`: (obsolete) defines functions used by the Virtual Memory Engine.
  - `loader.h`: (obsolete) defines functions used by the loader, which loads the program from disk into memory.
  - `common.h`: defines structs and functions used throughout the OS.
  - `bitopts.h`: defines operations on bit data.
  - `os-mm.h, mm.h, mm64.h`: define the structures and basic data for paging-based memory management.
  - `syscall.h`: defines system call headers.
  - `libxxx.h`: (Optional) defines the standard library headers.
  - `os-cfg.h`: (Optional) defines the constants used to switch software configurations.
- Source files
  - `timer.c`: implements the timer.
  - `cpu.c`: implements the virtual CPU.
  - `queue.c`: implements operations on (priority) queues.
  - `paging.c`: (obsolete) is used to check the functionality of the Virtual Memory Engine.
  - `dma*`: DMA engines.
  - `os.c`: contains the main function that starts the whole OS.
  - `loader.c`: implements the loader.

  - `sched.c`: implements the scheduler.
  - `mem.c`: (obsolete) implements the previous RAM and virtual memory design.
  - `mm.c, mm64.c, mm-vm.c, mm-memphy.c`: implement paging-based memory management.
  - `libmem.c, libstd.c`: implement the standard library.
  - `syscall.c, syscall.tbl, syscalltbl.sh, sys_xxx.c`: implement system calls.

- `Makefile`
- `input`: the folder contains a set of inputs used for verification.
- `output`: sample outputs of the system.

1.3 Processes

We are going to build a multitasking OS that lets multiple processes run concurrently, so it is worth spending some space explaining the organization of processes. The OS manages processes through their PCB, described as follows:

```
// From include/common.h
struct pcb_t {
     uint32_t pid;
     uint32_t priority;
     char path[100];
     uint32_t code_seg_t * code;
     addr_t regs[10];
     uint32_t pc;
#ifdef MLQ_SCHED
     uint32_t prio;
#endif
     struct page_table_t * page_table;
     uint32_t bp;
}
```

The meaning of fields in the struct:

- **PID**: the process ID.
- **priority**: process priority; the lower the value, the higher the priority. This legacy priority depends on the process's properties and remains fixed throughout execution.
- **code**: Text segment of the process (To simplify the simulation, we do not put the text segment in RAM).
- **regs**: Registers, each process could use up to 10 registers numbered from 0 to 9.
- **pc**: the current value of the program counter.
- **page_table**: the translation from virtual addresses to physical addresses (obsolete, do not use).
- **bp**: the break pointer, used to manage the heap segment.
- **prio**: Priority on execution (if supported), and this value overwrites the default priority.

Similar to a real process, each process in this simulation is just a list of instructions executed by the CPU one by one from beginning to end (we do not implement jump instructions here). There are five instructions that a process can perform:

- **CALC**: performs some calculation using the CPU. This instruction does not take any arguments.

  ***Annotation of Memory region**: A memory region is a storage area allocated for a variable. This term is associated with an index in the SYMBOL TABLE and is usually made human-readable through a variable name and a mapping mechanism. Unfortunately, this mapping is outside the scope of this Operating Systems course. It belongs more naturally to a compiler course, which explains how the compiler and loader do their jobs and map labels to the associated indices. For simplicity, we refer to a memory region by its index, and there is a limit on the number of variables in each program/process.*

- **ALLOC**: Allocate some chunk of bytes on the main memory (RAM). Instruction's syntax:

```
alloc [size] [reg]
```

  where **size** is the number of bytes the process wants to allocate from RAM, and **reg** is the register number that will store the address of the first byte of the allocated memory region. For example, the instruction **alloc 124 7** allocates 124 bytes from the OS, and the address of the first of those 124 bytes will be stored in register #7.

- **KMALLOC** Allocate some chunk of bytes on the kernel memory space. Instruction's syntax:

```
kmalloc [size] [reg]
```

  where **size** is the number of bytes the process wants to allocate from RAM, and **reg** is the register number that will store the address of the first byte of the allocated memory region. It allocates physically contiguous memory. The OS must find a contiguous block of physical memory.

- **FREE** Free allocated memory. Instruction's syntax:

```
free [reg]
```

  where **reg** is the number of registers holding the address of the first byte of the memory region to be deallocated.

- **READ (userspace only)** Read a byte from memory and can only access userspace address. It prevents the supervisor mode access to kernelspace address. Instruction's syntax:

```
read [source] [offset] [destination]
```

  The instruction reads one byte memory at the address which equal to the value of register **source** + **offset** and saves it to **destination**. For example, assume that the value of register #1 is **0x123** then the instruction **read 1 20 2** will read one byte memory at the address of **0x123 + 14** (14 is 20 in hexadecimal) and save it to register #2.

- **WRITE (userspace only)** Write a value register to memory and can only access userspace address. It prevents the supervisor mode access to kernelspace address. Instruction's syntax:

```
write [data] [destination] [offset]
```

  The instruction writes **data** to the address which equal to the value of register **destination** + **offset**. For example, assume that the value of register #1 is **0x123** then the instruction **write 10 1 20** will write 10 to the memory at the address of **0x123 + 14** (14 is 20 in hexadecimal).

- **COPY_FROM_USER** do a direct read from the userspace address and write to the kernelspace address. Instruction's syntax:

```
copy_from_user [source] [destination] [offset] [size]
```

The instruction performs data transfer from user space to kernel space. It reads data from **source** and write to **destination** starting from given **offset** with **size**

- **COPY_TO_USER** do a direct read from the kernelspace address and write to the userspace address. Instruction's syntax:. Instruction's syntax:

```
copy_to_user [source] [destination] [offset] [size]
```

The instruction performs data transfer from kernel space to user space. It reads data from **source** and writes to **destination** starting from given **offset** with **size**.

- **DMA_MEMCPY** do a direct copy memory content from source to destination regions at the same offset. Instruction's syntax:

```
dma_memcpy [source] [destination] [offset] [size]
```

The instruction performs data transfer from source to destination. It reads data from **source** and writes to **destination** starting from given **offset** with **size**.

1.4 How to Create a Process?

The content of each process is actually a copy of a program stored on disk. Thus to create a process, we must first generate the program which describes its content. A program is defined by a single file with the following format:

```
[priority] [N = number of instructions]
instruction 0
instruction 1
...
instruction N-1
```

where **priority** is the **default** priority of the process created from this program. It needs to remind that this system employs a dual priority mechanism.
The higher priority (with the smaller value) the process has, the process has higher chance to be picked up by the CPU from the queue (See section 2.1 for more detail). **N** is the number of instructions and each of the next **N** lines(s) are instructions represented in the format mentioned in the previous section. You could open files in **input/proc** directory to see some sample programs.
Dual priority mechanism Please remember that this default value can be overwrite by the *live* priority during process execution calling. For tackling the conflict, when it has priority in process loading (this inputt file), it will overwrite and replace the default priority in process description file.
The dual mode is common in modern OS where a program can be separated with various priorities from low important user work to high importane system daemon. But, in a specific context, user can execute the program with a priority different from the default (right click to change the priority of running process in Task Manager or run an application with Administration privilege).

1.5 How to Run the Simulation

What we are going to do in this assignment is to implement a simple OS and simulate it over virtual hardware. To start the simulation process, we must create a description file in **input** directory about the hardware and the environment that we will simulate. The description file is defined in the following format:

```
[time slice] [N = Number of CPU] [M = Number of Processes to be run]
[time 0] [path 0] [priority 0]
[time 1] [path 1] [priority 1]
```

```
    ...
    [time M-1] [path M-1] [priority M-1]
```

where **time slice** is the amount of time (in seconds) for which a process is allowed to run. **N** is the number of CPUs available and `M` is the number of processes to be run. The last parameter **priority** is the *live* priority when the process is invoked; it overwrites the default priority in the process description file (see Section 1.4). To start the simulation, you must compile the source code first by using **make all** command. After that, run the command

```
    ./os [configure_file]
```

where **configure_file** is the path to configure file for the environment on which you want to run and it should associated with the name of a description file placed in **input** directory.

1.6 How to Write the Kernel Interface

1.6.1 The kernel structure

The `struct krnl_t` encapsulates key components of the operating system kernel for managing process scheduling and memory subsystem. It integrates memory management units and physical memory abstractions, enabling efficient handling of multiple RAM and swap spaces.

```
// From include/common.h
/* Kernel structure */
struct krnl_t
{
    struct queue_t *ready_queue;
    struct queue_t *running_list;
#ifdef MLQ_SCHED
    struct queue_t *mlq_ready_queue;
#endif
#ifdef MM_PAGING
    struct mm_struct *mm;
    struct memphy_struct *mram;
    struct memphy_struct **mswp;
    struct memphy_struct *active_mswp;
    uint32_t active_mswp_id;
#endif
};
```

1.6.2 The system call

The system call is the fundamental interface between an application and the Simple Operating System kernel.

System calls and library wrapper functions System calls are generally not invoked directly, but rather through wrapper functions in `libstd` (or perhaps other libraries). The `libstd` wrapper functions are usually quite thin, doing little work other than copying arguments to the appropriate registers before invoking the system call. The details of direct system-call invocation are illustrated in Figure 2.

System call list The list of system calls that are available in the Simple Operating System is shown in the following listing:

```
System call       Kernel   Notes
-----------------------------------------------
listsyscall       3.0
memmap            3.0
```

System call manual page

1. **listsyscall** lists all system calls.

```
Name
  listsyscall - list all system calls
Synopsis
  SYSCALL 0
Description
  listsyscall displays the list of all system calls.
```

2. **memmap** maps memory.

```
Name
  memmap - map memory
Synopsis
  SYSCALL 17 SYSMEM_OP REG_ARG2 REG_ARG3
Description
  memmap supports various memory mapping operations, including:
  + SYSMEM_MAP_OP with vmap_pgd_memset() handler
  + SYSMEM_INC_OP with inc_vma_limit() handler
  + SYSMEM_SWP_OP with __mm_swap_page() handler
  + SYSMEM_IO_READ with MEMPHY_read() handler
  + SYSMEM_IO_WRITE with MEMPHY_write() handler
```

Question What are the advantages and disadvantages of using the unified system call interface for manipulating different system components, i.e. read/write/free for files, memory and I/O devices. In your analysis, consider how this abstraction influences operating system design, performance trade-offs, error handling complexity, and the balance between portability and efficiency.

1.6.3 Adding a system call to Simple Operating System

In this guide, you will learn how to add a simple system call to the Simple Operating System.

Step 1- Creation

1. Create a C file for your system call in `src/sys_xxxhandler.c`

```c
// From src/sys_xxxhandler.c
#include "common.h"
#include "syscall.h"
#include "stdio.h"

int __sys_xxxhandler(struct pcb_t *caller, struct sc_regs* reg)
{
   /* TODO: implement syscall job */
   printf("The first system call parameter %d\n", regs->a1);
   return 0;
}
```

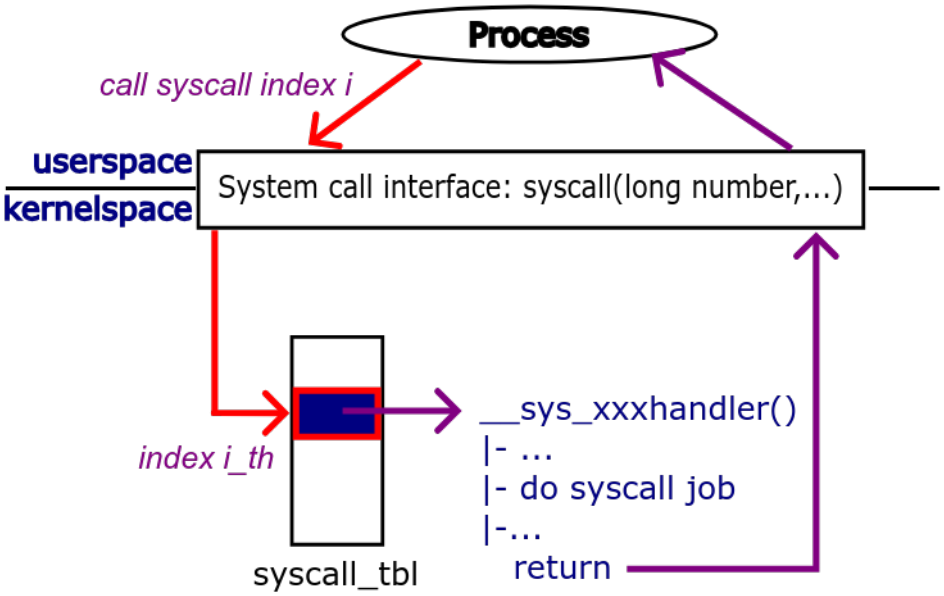


Figure 2: The handling of user application invoking a system call

2. Create a Makefile entry for your system call with a unique indexing, i.e. 440

```
# From Makefile
SYSCALL_OBJ += $(addprefix $(OBJ)/, sys_xxxhandler.o)
```

3. Add your system call to the kernel's system call table

```
# From src/syscall.tbl
440     xxx         sys_xxxhandler
```

Step 2- Installation In this section, you will install the new kernel and prepare your Simple Operating System to boot into it

1. Compile the kernel source code

```
$make all
```

2. Boot your Simple Operating System

```
$./os <input_file>
```

Final step - Checking results In this section, you will write a program to check whether your system call works or not. After that, you will see your system call in action.

1. Create a program **sc** to invoke your system call

```
20 1
syscall 440 1
```

2. A create a configuration file **os_syscall** to boot your Simple Operating System

```
2 1 1
2048 16777216 0 0 0
9 sc  15
```

3. Boot your Simple Operating System

```
$./os os_syscall
```

4. Check the last line of the output messages

```
...
The first system call paramter 1
     CPU 0: Processed  1 has finished
     CPU 0 stopped
```

Congratulations! You have successfully added a system call to the Simple Operating System!

Question When a system call executes too long time, how does the Operating system detect and handle the case?

2 Implementation

2.1 Scheduler

We first implement the scheduler. Figure 3 shows how the operating system schedules processes. The OS is designed to work on multiple processors. The OS uses multiple queue called **ready_queue** to determine which process to be executed when a CPU becomes available. Each queue is associated with a fixed priority value. The scheduler is designed based on "multilevel queue" algorithm used in Linux kernel[1].

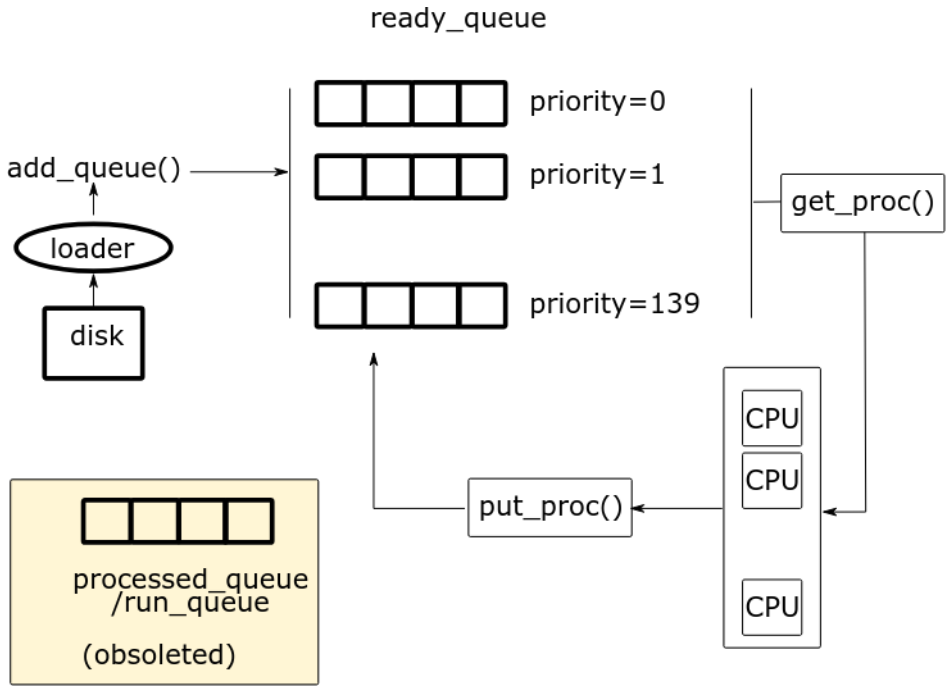


Figure 3: The operation of scheduler in the assignment

According to Figure 3, the scheduler works as follows. For each new program, the loader will create a new process and assign a new PCB to it. The loader then reads and copies the content of the program to the text segment of the new process (pointed by **code** pointer in the PCB of the process - section 1.3). The PCB of the process is pushed to the associated **ready_queue** having the same priority with the value *prio* of this process. Then, it waits for the CPU. The CPU runs processes in round-robin style. Each process is allowed to run in time slice. After that, the CPU is forced to enqueue the process back to it associated **priority ready_queue**. The CPU then picks up another process from **ready_queue** and continue running.

In this system, we implement the Multi-Level Queue (MLQ) policy. The system contains **MAX_PRIO** priority levels. Although the real system, i.e. Linux kernel, may group these levels into subsets, we keep the design where each priority is held by one **ready_queue** for simplicity. We simplify the add_queue and **put_proc** as putting the proc to appropriated ready queue by priority matching. The main design is belong to the MLQ policy deployed by **get_proc** to fetch a proc and then dispatch CPU.

The description of **MLQ policy**: the traversed step of **ready_queue list** is a fixed formulated number based on the priority, i.e. slot= (MAX_PRIO - prio), each queue have only fixed slot to use the CPU and

[1]Actually, Linux supports the feedback mechanism which allow to move process among priority queues but we don't implement the feedback mechanism here

when it is used up, the system must change the resource to the other process in the next queue and left the remaining work for future slot even though it needs a completed round of ready_ queue.
An example in Linux MAX_PRIO=140, prio=0..(MAX_PRIO - 1)

```
prio = 0        |         1          | ....    | MAX_PRIO - 1
slot = MAX_PRIO |   MAX_PRIO - 1     | ....    |      1
```

MLQ policy only goes through the fixed step to traverse all the queue in the priority `ready_queue` list. Your job in this part is to implement this algorithm by completing the following functions

- `enqueue() and dequeue()` (in `queue.c`): We have defined a struct (`queue_t`) for a priority queue at `queue.h`. Your task is to implement those functions to help put a new PCB to the queue and get the next 'in turn' PCB out of the queue.
- `get_proc()` (in `sched.c`): gets PCB of a process waiting from the `ready_queue` system. The selected `ready_queue` 'in turn' has been described in the above policy.

You could compare your result with model answers in ***output*** *directory. Note that because the loader and the scheduler run concurrently, there may be more than one correct answer for each test.*
Note: the `run_queue` is something not compatible with the theory and has been obsoleted for a while. We don't need it in both theory paradigm and code implementation, it is such a legacy/outdated code but we still keep it to avoid bug tracking later.
Question: Considering the impactness of MLQ detailed policies, what is the benefit of each policy?

2.2 Memory Management

2.2.1 The process memory layout

The virtual memory space is organized as a memory mapping for each process PCB. From the process point of view, the virtual address includes multiple **vm_area**s (contiguously). In the real world, each area can act as code, stack or heap segment. Therefore, the process keeps in its **pcb** a pointer of multiple contiguous memory areas.

Memory Area Each memory area ranges continuously in [**vm_start,vm_end**]. Although the space spans the whole range, the actual usable area is limited by the top pointing at `sbrk`. In the area between **vm_start** and **sbrk**, there are multiple regions captured by **struct vm_rg_struct** and free slots tracking by the list **vm_freerg_list**. Through this design, we make the design to perform the actual allocation of physical memory only in the usable area, as in Figure 4.

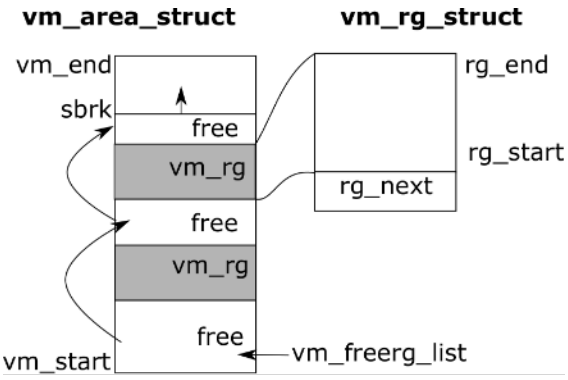


Figure 4: The structure of vm area and region

```
//From include/os-mm.h
/*
 *  Memory region struct
 */
struct vm_rg_struct {
   unsigned long rg_start;
   unsigned long rg_end;

   /* Priviledge mode bit
      usermode   - mode bit = 1
      kernelmode - mode bit = 0
   */
   unsigned long mode_bit;

   struct vm_rg_struct *rg_next;
};

/*
 *  Memory area struct
 */
struct vm_area_struct {
   unsigned long vm_id;
   unsigned long vm_start;
   unsigned long vm_end;

   unsigned long sbrk;
/*
 * Derived field
 * unsigned long vm_limit = vm_end - vm_start
 */
   struct mm_struct *vm_mm;
   struct vm_rg_struct *vm_freerg_list;
   struct vm_area_struct *vm_next;
};
```

Memory region As we noted in the previous section 1.3, these regions are actually acted as the variables in the human-readable program's source code. Due to the current out-of-scope fact, we simply touch in the concept of namespace in term of indexing. We have not been equipped enough the principle of the compiler. It is, again, overwhelmed to employs such a complex symbol table in this OS course. We temporarily imagine these regions as a set of limit number of region. We manage them by using an array of symrgtbl[PAGING_MAX_SYMTBL_SZ]. The array size is fixed by a constant, PAGING_MAX_SYMTBL_SZ, denoted the number of variable allowed in each program. To wrap up, we use the `struct vm_rg_struct symrgtbl` to keep the start and the end point of the region and the pointer `rg_next` is reserved for future set tracking.

```c
//From include/os-mm.h
/*
 * Memory mapping struct
 */
struct mm_struct {
   uint32_t *pgd;

   struct vm_area_struct *mmap;

   /* Currently we support a fixed number of symbol */
   struct vm_rg_struct symrgtbl[PAGING_MAX_SYMTBL_SZ];

   struct pgn_t *fifo_pgn;

   /* kmem cache pool */
   struct kcache_pool_struct *kcpooltbl;
};
```

Canonical address layout In a 64-bit system, the hardware typically implements only 48-bit or 57-bit addressing rather than the full 64-bit range. Historically, some software designs misused the unused higher-order bits for purposes other than memory addressing, which led to inconsistencies. To prevent this, modern architectures enforce a rule where all unused bits must be fixed to either 0 or 1, creating what is known as a canonical address. This ensures that every valid memory address follows a standardized, reliable format.

In the appendices, we have some various address layouts of baseline OS e.g. UNIX-like on page 29, NTkernel on page 32 etc. We use the Linux memory layout as a part of UNIX-like OS in the CO2018 lab environment. However, for application convenience, we encourage students to implement a macOS-style memory layout this semester, as the hardware engine accelerates memory operations. This approach is also suitable for managing **spearated small, medium, and large memory regions**, following the macOS memory layout model on page 31 .

Basic memory operations in kernel memory in this semester work, we dont focus on kernel memory management, skip it simple as a continous memory allocation strategy.
However, we still require privileged memory access restrictions between userspace and kernel space. A user program can read and write user-space addresses, but kernel addresses are prohibited. Attempting to access kernel memory results in invalid memory operations. Therefore, it must use functions such as `copy_from_user` and `copy_to_user` to transfer data between kernel space and user space before the data can be accessed.

2.2.2 The virtual memory mapping in each process

Memory mapping is represented by `struct mm_struct`, which tracks all the mentioned memory regions in a separated contiguous memory area. In each memory mapping struct, many memory areas are pointed out by `struct vm_area_struct *mmap` list. An important field is the `pgd`, which is the page table directory and contains all page table entries. Each entry maps the page number to the frame number in the paging memory management system. We provide a detailed page-frame mapping in the later section 2.2.4. The `symrgtbl` is a simple implementation of the symbol table. The other fields are mainly used to track specific user operations i.e. caller, fifo page (for referencing). We have included them for your use; you can utilize them as needed (or discard).

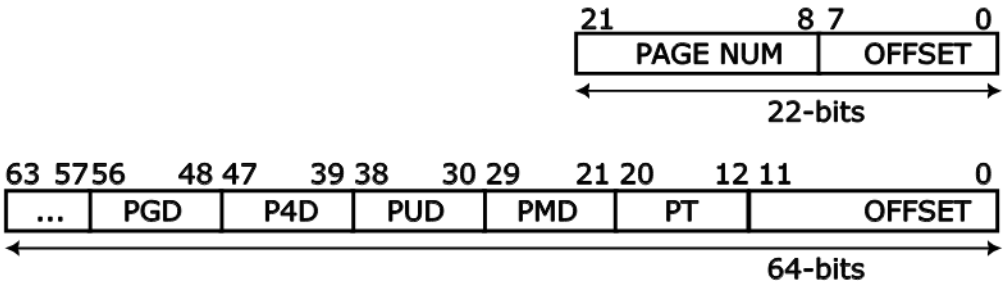


Figure 5: CPU address scheme

CPU 32-bit address scheme the address generated by CPU to access a specific memory location. In paging-based system, it is divided into:

- **Page number (p)**: used as an index into a page table that holds the based address for each page in physical memory.
- **Page offset (d)**: combined with base address to define the physical memory address that is sent to the Memory Management Unit

The physical address space of a process can be non-contiguous. We divide physical memory into fixed-sized blocks (the frames) with two sizes 256B or 512B. We proposed various setting combinations in Table 1 Based on the configuration of 22-bit CPU and 256B page size, the CPU address is organized as in Figure 5.

CPU bus	PAGE size	PAGE bit	No pg entry	PAGE Entry sz	PAGE TBL	OFFSET bit	PGT mem	MEMPHY	fram bit
20	256B	12	~4000	4byte	16KB	8	2MB	1MB	12
22	256B	14	~16000	4byte	64KB	8	8MB	1MB	12
22	512B	13	~8000	4byte	32KB	9	4MB	1MB	11
22	512B	13	~8000	4byte	32KB	9	4MB	128kB	8
16	512B	8	256	4byte	1kB	9	128K	128kB	4

Table 1: Various CPU address bus configurations

CPU 64-bit address scheme The 64-bit 5-level paging scheme is applied on CPU-64 bit. It expands the virtual address space up to 128 petabytes (PiB) of virtual memory

- Page level 5, Page Global Directory (PGD): bit 56-48
- Page level 4, Page Level 4 Directory (P4D): bit 47-39
- Page level 3, Page Upper Directory (PUD): bit 38-30
- Page level 2, Page Middle Directory (PMD): bit 29-21
- Page level 1, Page Table (PT): bit 20-12
- Page offset: bit 11-0

Bit	63-57	56-48	47-39	38-30	29-21	20-12	11-0
Paging level	—	L5	L4	L3	L2	L1	OFFSET
Page directories	—	PGD	P4D	PUD	PMD	PT	OFFSET
Memory size	—	128PiB	256TB	512GB	1GB	2MB	4KB

Table 2: CPU 64-bit scheme

An example of 5-level paging address scheme

Level 1, Page Table (PT): Index = (0x39ffe1d9c9000 >> 12) & 0x1ff = 0x1c9

Level 2, Page Middle Directory (PMD): Index = (0x39ffe1d9c9000 >> 21) & 0x1ff = 0x0ec

Level 3, Page Upper Directory (PUD): Index = (0x39ffe1d9c9000 >> 30) & 0x1ff = 0x1f8

Level 4, Page Level 4 Directory (P4D): Index = (0x39ffe1d9c9000 >> 39) & 0x1ff = 0x13f

Level 5, Page Global Directory (PGD): Index = (0x39ffe1d9c9000 >> 48) & 0x1ff = 0x03

Multi-level page table size page table size for 64-bit address scheme would be critical large, there are many strategies to improve the page table size, including:

- Sparse allocation with hashing: page tables are created only for virtual address regions that are actually used.
- Demand (dynamically) allocation of page tables: only create pagetable entries for pages that are actively being used.

Students are encouraged to make design decisions through a careful comparison the trade-off between page table traversal time, the number of member accesses and the required storage space.
In the VM summary, all structures supporting VM are placed in the module `mm-vm.c` and `mm.c`. The 64-bit address support is placed in `mm64.c`
Question: What is the primary motivation for combining segmentation with paging in memory management? How does this hybrid approach address limitations inherent in using either technique alone?

2.2.3 The system's physical memory

Figure 1 shows that the memory hardware is installed in terms of the whole system. All processes own their separated memory mappings, but all mappings target a singleton physical device. There are two types of devices, RAM and SWAP. They both can be implemented by the same physical device as in `mm-memphy.c` with different settings. The supported settings are random memory access, sequential/serial memory access, and storage capacity.

Despite the various possible configurations, the logical use of these devices can be distinguished. The RAM device, which belongs to the primary memory subsystem, can be accessed directly from the CPU address bus, allowing it to be read or written using CPU CPU instructions. Meanwhile, SWAP is just a secondary memory device, and all data manipulation must be performed by moving them to the main memory. Since it lacks direct access from the CPU, the system usually equips a large SWAP at a low cost and may have more than one instance. In our settings, we support hardware configured with one RAM device and up to 4 SWAP devices.

The `struct framephy_struct` is mainly used to store the frame number.
The `struct memphy_struct` has basic fields storage and size. The `rdmflg` field defines whether the memory access is random or sequential. The fields `free_fp_list` and `used_fp_list` are reserved for retaining unused and used memory frames, respectively.

```
//From include/os-mm.h
/*
 * FRAME/MEM PHY struct
 */
struct framephy_struct {
   int fpn;
   struct framephy_struct *fp_next;
};
```

```
struct memphy_struct {
   /* Basic field of data and size */
   BYTE *storage;
   int maxsz;

   /* Sequential device fields */
   int rdmflg;
   int cursor;

   /* Management structure */
   struct framephy_struct *free_fp_list;
   struct framephy_struct *used_fp_list;
};
```

Question: What are the benefits of extending hierarchical paging to N level?

2.2.4 Paging-based address translation scheme

The translation supports both segmentation and segmentation with paging. In this version, we develop a single-level paging system that leverages one RAM device and one SWAP instance hardware. We have implemented the capability to handle multiple memory segments, but we mainly focus on the first segment of vm_area (vmaid = 0). The further versions will take into account a sufficient paging scheme for multiple segments and the potential overlap/non-overlap between segments.

Page table entry CPU 32-bit & 64-bit As in Figure 6

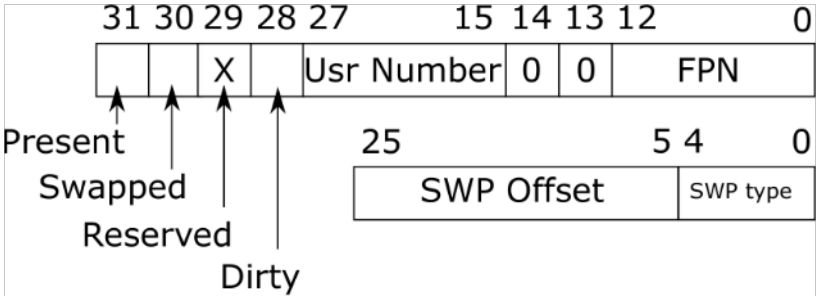


Figure 6: Page Table Entry Format.

This structure allows a userspace process to determine which physical frame each virtual page is mapped to. It contains a 32-bit value for each virtual page, containing the following data:

```
    * Bits 0-12  page frame number (FPN) if present
    * Bits 13-14 zero if present
    * Bits 15-27 user-defined numbering if present
    * Bits 0-4   swap type if swapped
    * Bits 5-25  swap offset if swapped
    * Bit  28    dirty
    * Bits 29    reserved
    * Bit  30    swapped
    * Bit  31    presented
```

Page table The virtual space is isolated for each entity, so each **struct pcb_t** has its own table. To work in a paging-based memory system, we need to update this struct, and the next section discusses the required modifications. In all cases, each process has a completely isolated and unique space; therefore, **N** processes in our setting result in **N** page tables. Each page table must contain entries for the entire CPU address space. For each entry, the page number may have an associated frame in MEMRAM or MEMSWP, or it may have a null value. In the highlighted setting shown in Table 1, we have a 16,000-entry table, and each table costs 64 KB of storage space.

In Section 2.2.2, the process can access virtual memory contiguously through the vm area structure. The remaining work deals with the mapping between pages and frames to provide contiguous memory space over a discrete frame storage mechanism. This falls into two main areas: memory swapping and basic memory operations, i.e., alloc/free/read/write, which mostly interact with the **pgd** page table structure.

Memory swapping We have noted that a memory area (/segment) may not use all of its available storage space. This means that some storage remains unmapped to MEMRAM. Swapping helps move the contents of a physical frame between MEMRAM and MEMSWAP. Swapping in copies a frame's content from secondary storage to main memory (RAM). Swapping out, conversely, attempts to move the content of a frame in MEMRAM to MEMSWAP. In a typical context, swapping helps free RAM frames because the swap device is usually large enough.

Basic memory operations in paging-based system

- ALLOC: the user calls library functions in `libmem`, and in most cases the allocation fits into the data segment area. If there is no suitable space, we need to expand memory by raising the barrier set by `sbrk`. Since that space may never have been touched, the system may need to use MMU system calls to obtain physical frames and then map them using page table entries.
- FREE: the user calls library functions in `libmem` to revoke the storage space associated with the given region ID. Since we cannot reclaim the occupied physical frame without potentially causing memory holes, we keep the released storage space in a free list for future allocation requests; this logic is embedded in the `libmem` library.
- READ/WRITE: these operations require the page to be present in main memory. The most resource-consuming step is page swapping. If the page is in the MEMSWAP device, it needs to be brought back to the MEMRAM device (swapping in), and if there is not enough space, we need to move some pages back to the MEMSWAP device (swapping out) to make room.

To perform these operations, collaboration among the `mm` modules is needed, as illustrated in Figure 7.

Advanced topics Due to the inherently non-contiguous space of 64-bit page tables, students are free to implement a unified access model if they are confident in programming address translation. Alternatively, they can separate and map it cascadedly into two different page tables: one for user space and another for kernel space. However, it is important to remember the theoretical requirement that kernel space must remain identical for all processes.
Question What are the advantages and disadvantages of paging and contiguous memory allocation?

2.2.5 Wrapping-up all paging-oriented implementations

Introduction to configuration control using constant definitions: [2] To reduce the effort required to handle interference among feature-oriented program modules, we apply a common software-engineering

[2] This section is applied mainly to paging memory management. If you are still working on the scheduler section, you should keep the default settings and avoid making too many changes to these values.

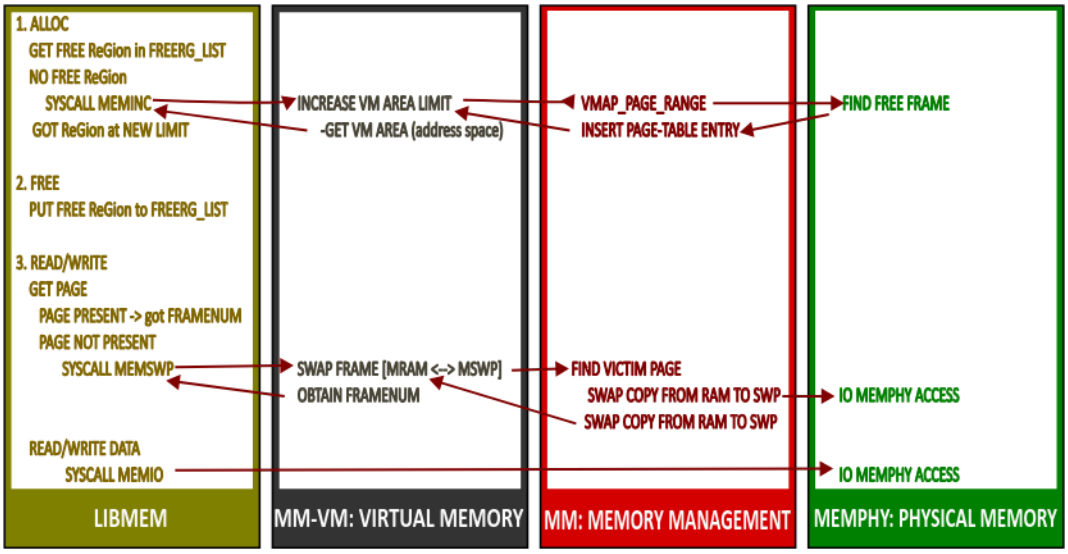


Figure 7: Memory system modules

approach by isolating each feature through a configuration system. Leveraging this mechanism, we can maintain various subsystems separately while keeping them in a single codebase. We can control the configuration used in our simulation program in the `include/os-cfg.h` file.

```
// From include/os-cfg.h
#define MLQ_SCHED 1
#define MAX_PRIO 140

#define MM_PAGING
#define MM_FIXED_MEMSZ
```

An example of the `MM_PAGING` setting: With this new paging module, we derive the PCB struct by adding some additional memory-management fields, and they are wrapped by a constant definition. If we want to use the `MM_PAGING` module, we enable the associated `#define` configuration line in `include/os-cfg.h`.

```
// From include/common.h
struct pcb_t {
     ...
#ifdef MM_PAGING
     struct mm_struct *mm;
     struct memphy_struct *mram;
     struct memphy_struct **mswp;
     struct memphy_struct *active_mswp;
#endif
     ...
};
```

Another example of the `MM_FIXED_MEMSZ` setting: With the new version of the PCB struct, the description file in `input` can keep the old setting with `#define MM_FIXED_MEMSZ` while still working in the new paging memory-management mode. This configuration provides backward compatibility with older input files.

New configuration with an explicit declaration of memory sizes (Be careful: this mode supports custom memory sizes, which implies that we comment out, delete, or disable the constant `#define MM_FIXED_MEMSZ`). In this mode, the simulation program takes one additional line from the input file. This line contains the system's physical memory sizes: one MEMRAM and up to four MEMSWP devices. Each size value must be a non-negative integer. We can set a size equal to 0, but that means the corresponding swap device is deactivated. To keep the parameters valid, we must have one MEMRAM and at least one MEMSWAP; those values must be positive integers, while the remaining values can be set to 0. The last configuration value is the maximum size of virtual memory, which is a prerequisite for the heap-grow-down setting.

```
[time slice] [N = Number of CPU] [M = Number of Processes to be run]
[KERNEL_BASE_ADDR]
[RAM_SZ] [SWP_SZ_0] [SWP_SZ_1] [SWP_SZ_2] [SWP_SZ_3]
[time 0] [path 0] [priority 0]
[time 1] [path 1] [priority 1]
...
[time M-1] [path M-1] [priority M-1]
```

The highlighted input line is controlled by the constant definition. Double check the input file and the contents of `include/os-cfg.h` will help us understand how the simulation program behaves when there may be something strange.

Configuration of MM64 defines the memory-management structure for a system supporting a 64-bit memory scheme.

KERNEL_BASE_ADDR supports a dynamic floating kernel space address range. If this is not available, This base layout is determined by a specific configuration is defined as shown in the Table **??**.

```
// From include/os-mm.h
/* Memory management struct */
struct mm_struct {
#ifdef MM64
    uint64_t *pgd;
    uint64_t *p4d;
    uint64_t *pud;
    uint64_t *pmd;
    uint64_t *pt;
#else
    uint32_t *pgd;
#endif
    ...
}
```

2.3 Co-processors

2.3.1 Asynchronous operation and bus address

Asynchronous operation is illustrated as in Figure. 8, where the CPU initiates an I/O request and then continues executing other instructions instead of waiting for the I/O to finish. When the I/O controller completes the operation or encounters an event, it generates an interrupt (Step 4). The CPU temporarily suspends the current task, executes the interrupt handler to process the I/O result. This approach improves CPU utilization by performing computation and I/O operations asynchronously.

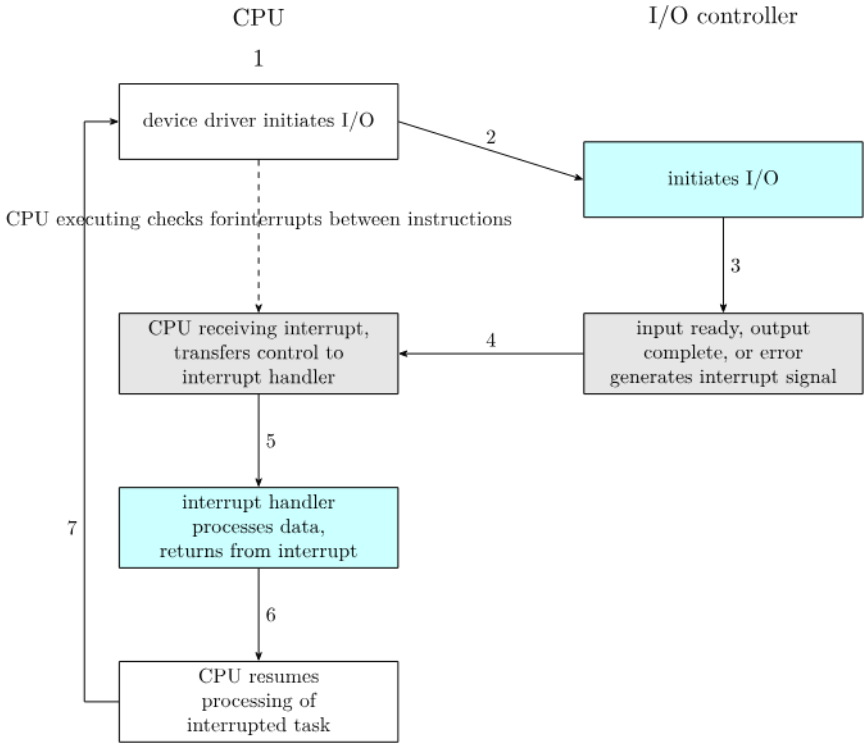


Figure 8: Interrupt-driven I/O

Figure. 9 presents the operation of a modern computer, in which one or more CPU repeatedly cores repeatedly fetch instructions and data from memory through the cache, execute them, while peripheral devices perform I/O asynchronously. The CPU and co-processors exchange data either through CPU-controlled transfers or directly with memory via **bus address**. Co-processor notifies the CPU via interrupts when service is required.

2.3.2 Direct Memory Access - DMA

DMA is a dedicated hardware unit that manages data transfers between computer memories. DMA can also be used for memory-to-memory transfers. This feature is useful at any time that the CPU cannot keep up with the rate of data transfer, or when the CPU needs to perform work while waiting for a data transfer (*refers Wikipedia*[3]). The principles of data transfer can be sketched as follows:

- **The problem**: Without DMA, the CPU must manually read every byte of data from a peripheral into a register, and then write it back into RAM (a process called Programmed I/O).
- **The co-processor solution**: The CPU tells the DMA controller the source, destination, and size of the data transfer, and then goes back to executing other programs. The DMA controller takes over the system bus, moves the data directly, and notifies the CPU only when the entire transfer is finished.

Figure. 10 presents a comprehensive overview of the dma_top module, a configurable Direct Memory Access (DMA) controller designed for efficient, low-overhead data transfers. Table. 3 describes the complete pinout configuration required to integrate the DMA module into your system. In current settings, we use:

[3]https://en.wikipedia.org/wiki/Direct_memory_access

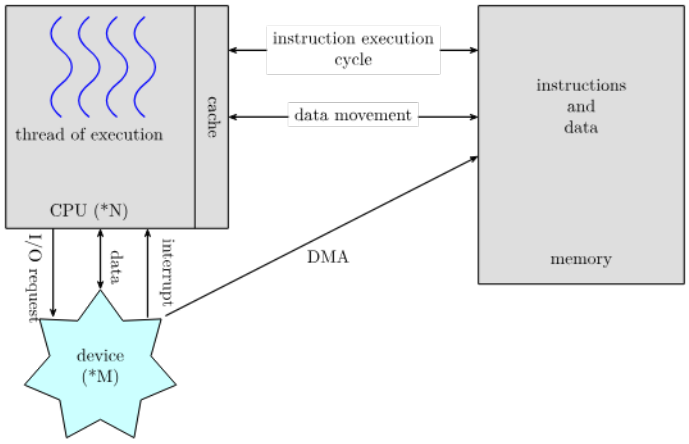


Figure 9: Modern computer works

- **ADDR_WIDTH**: 64 bits
- **DATA_WIDTH**: 32 bits

Due to the complexity of endianness mismatches, which involves the Application Binary Interface (ABI) at various levels (e.g., native fixed system rules or virtual with auto resolution), students are encouraged to process data byte-by-byte, even if the interface supports 32-bit data

Table 3: DMA module pin description

Signal Name	Direction	Width	Description
CLK	Input	1	Clock signal.
RST_N	Input	1	Reset.
IRQ	Output	1	Interrupt request line to the processor.
STATE	Output	2	Current operational state of the DMA module.
ch_en	Input	1	Channel enable signal to activate DMA transfers.
ch_pause	Input	1	Channel pause signal.
ch_done	Output	1	Asserted when the channel completes its transfer.
ch_error	Output	1	Asserted if an error occurs during transfer.
ch_xfer_count	Output	32	Total count of data transfers completed.
dat_valid	Input	1	Indicates that input data on lines is valid.
dat_0	Input/Output	ADDR_WIDTH	Address input line 0.
dat_1	Input/Output	ADDR_WIDTH	Address input line 1.
dat_2	Input/Output	DATA_WIDTH	Data input line 2.
dat_3	Input/Output	DATA_WIDTH	Data input line 3.
dat_ready	Output	1	DMA module readiness indication.

2.3.3 Graphic Processor Unit - GPU

Sharing the same asynchronous operations with other co-processors, GPUs are designed for massive parallelism which handles hundreds of thousands of independent mathematical operations at the exact same time. However, their primary focus is on graphics rendering and vector and matrix processing.

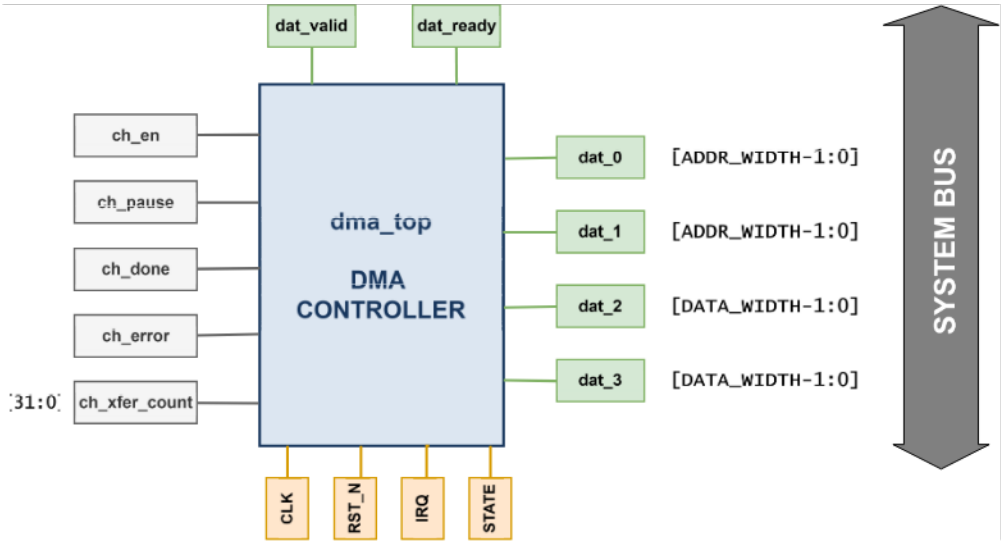


Figure 10: DMA co-processor

2.3.4 I/O acceleration technologies

These technologies leverage dedicated co-processors to accelerate I/O operations.

Memory operation offloading: offloads memory-to-memory data copies from the main processor to a dedicated chipset engine to free up CPU cycles.

Traffic and network optimization: moves network payloads directly to application memory without forcing the CPU to process each item.

Extended signaled interrupt: distributes I/O interrupts evenly across multiple chipset engines.

Low-latency interrupt: dynamically scales interrupt frequency based on the urgency

Streaming Accelerator: transfers streaming data

Pass through I/O: offloads memory copies and maximizes parallel data throughput.

2.4 Put It All Together

Finally, we combine the scheduler and memory management to form a complete OS. Figure 11 shows the complete organization of the OS memory-management system. The last task is synchronization. Since the OS runs on multiple processors, shared resources may be accessed concurrently by more than one process at a time. Your job in this section is to identify shared resources and use locking mechanisms to protect them. Check your work by first compiling the whole source code

```
make all
```

and compare your output with the samples in `output`. Remember that, because we are running in a multi-process environment, there may be more than one correct result. All outputs are provided only as samples and are not the only acceptable results. Your results only need to be explainable and comparable with the **theoretical framework**; they do not need to match the sample output exactly.

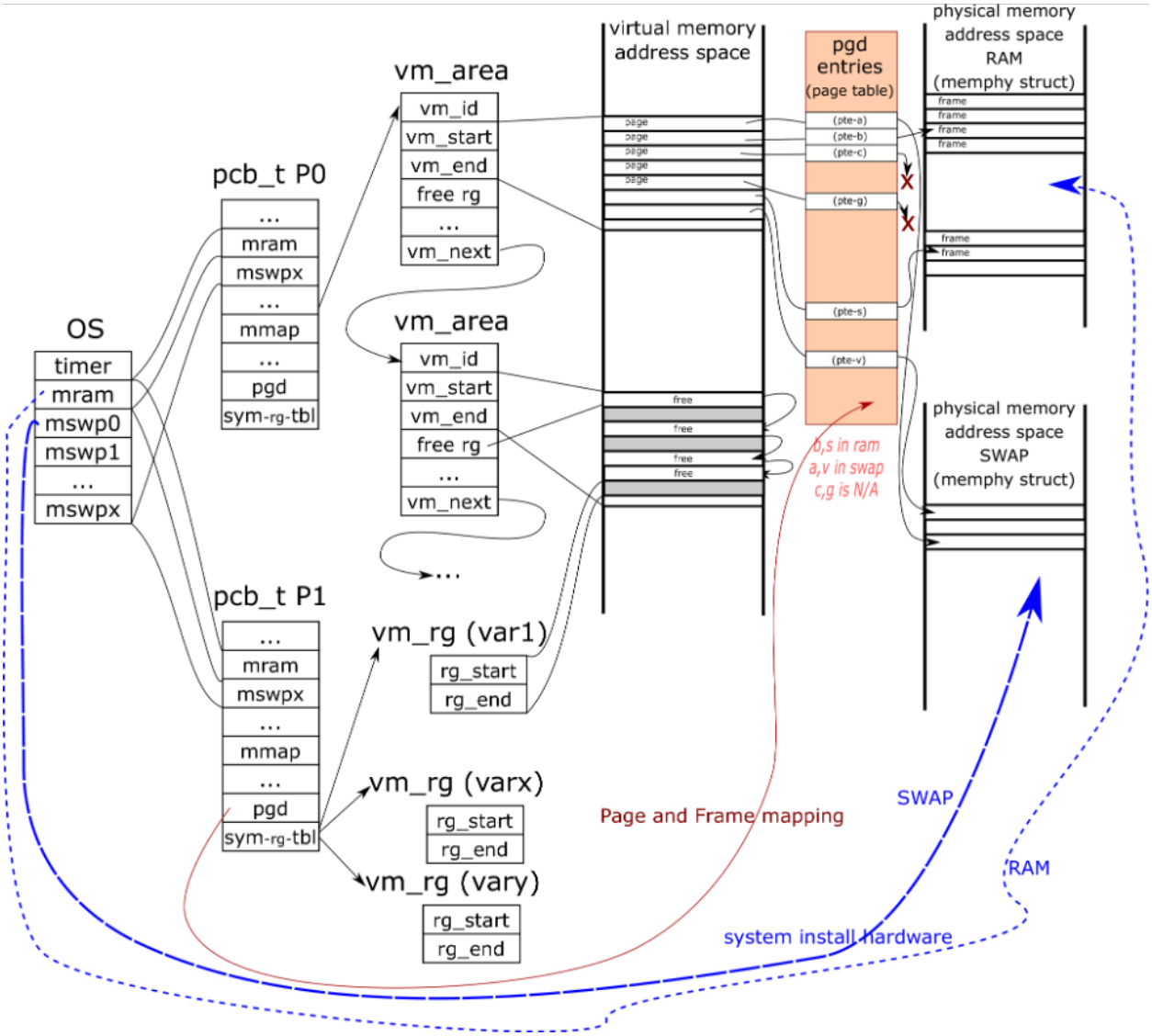


Figure 11: The operation related to virtual memory in the assignment

Question: What happens if the synchronization is not handled in your Simple OS? Illustrate the problem of your simple OS (assignment outputs) by example if you have any in the added kernel memory operations.

3 Submission

3.1 Source code

Requirement: You have to code the system calls following the coding style below. Reference: https://www.gnu.org/prep/standards/html_node/Writing-C.html

3.2 Requirements

Scheduler Implement the scheduler that employs the MLQ policy as described in Section 2.1.

Memory Management with separable user/kernel space Implement the paging subsystem.

****Note**:** the memory spaces are separated between user space and kernel space. Direct access from user space through a process PCB is not allowed; only `struct krnl_t` may be used.

Multi-level paging Implement the long-address 64-bit scheme with theoretical multi-level paging.

Optional page replacement Working with a 64-bit long-address scheme is a complex and demanding task. Therefore, the page-replacement requirements are defined by your lecturer. This decision is based on the complexity involved in implementing 64-bit memory management.

Students are encouraged to make design decisions through careful comparison of multiple strategies. In the report and output, students must present statistics on the number of memory accesses and the size of multilevel paging storage, and they must explain the benefits of their design.

Co-processor Implement the asynchronous interraction mechanism between CPU and co-processor.

All students: The DMA co-processor engine is provided. The appendices include a software prototype of the DMA engine (on page 33). Students may use this software prototype to develop and test a preliminary solution. However, since this semester's course is primarily intended for Computer Engineering students, all group projects and presentations must use the real DMA chip engine, supported by the hardware platform backed by `verilator`.

Talent students: You are encouraged to challenge yourself to design a new co-processor chip (GPU) with support for CUDA and the OpenVINO library, as both are primarily used for matrix and vector computations. Designing this chip will not earn you any extra points, as it is an opportunity to demonstrate your expertise. However, if you cannot complete the design, you can leave it as a challenge for your future career.

Questionnaire Students need to answer all questions in the assignment description.

3.3 Report

Write a report that answers questions in the implementation section and interprets the results of running tests in each section:

- Scheduling: draw a Gantt diagram describing how processes are executed by the CPU.
- Memory management: show the status of memory allocation in the data segments. You must ensure that communication between kernel space and user space through system calls uses PID passing, not direct PCB passing.
- Multi-level paging: show the multi-level paging address translation scheme.

- Overall: students should find their own way to interpret the simulation results.

After you finish the assignment, move your report to the source-code directory, compress the entire directory into a single file named `assignment_STUDENTID.zip`, and submit it to the LMS.

3.4 Grading

You must complete this assignment in groups of 4 or 5 students. The overall grade for your group is determined by the following components:

- Demonstration (7 points)
  - Scheduling: 2 points
  - MMU and user/kernel spaces: 2 points. This memory address work has an overlaps part with the address range of the next section, but it is counted in the next section points.
  - Co-processor: 3 points
    - 1 point the work of asynchronous operation
    - 2 points the work of bus addressing. One point concerns CPU access through the MMU, while the other point concerns co-processor access to memory.
- Report (3 points)

3.5 Code of ethics

Faculty staff members involved in code development reserve all copyright to the project source code.
Source Code License Grant: The author(s) hereby grant the Licensee personal permission to use and modify the Licensed Source Code for the sole purpose of studying while attending course CO2018 at HCMUT.

Revision History

Revision	Date	Author(s)	Description
1.0	01.2019	pdnguyen, Minh Thanh CHUNG, Hai Duc NGUYEN	Created CPU, scheduling, memory
1.1	09.2022	pdnguyen	Add Multilevel Queue (MLQ) CPU Scheduling
2.0	03.2023	pdnguyen	Initialize MM Paging Framework
2.1	10.2023	pdnguyen	Add Page Replacement
2.2	03.2024	pdnguyen	Add CPU Translation Lookaside Buffer (TLB)
2.3	10.2024	pdnguyen	Add heap segment
3.0	03.2025	pdnguyen	Add systemcall
4.0	10.2025	pdnguyen	Add 64-bit address
4.1	03.2026	pdnguyen	Add canonical address
5.0	10.2026	pdnguyen	Add co-processor

Appendix

Linux Address Layout

The layout is presented in the same order as the lab material, from low to high memory.

Start addr (hex)	**Offset**	**End addr (hex)**	**Size**	**VM area description**
0000000000000000	0	00ffffffffffffff	64 PB	user-space virtual memory, different per mm
0100000000000000	+64 PB	feffffffffffffff	~16K PB	huge, still almost 64 bits wide hole of non-canonical virtual memory addresses up to the -64 PB starting offset of kernel mappings.
Kernel-space memory, shared between all processes				
ff00000000000000	-64 PB	ff0fffffffffffff	4 PB	guard hole / reserved for hypervisor
ff10000000000000	-60 PB	ff10ffffffffffff	0.25 PB	LDT remap for PTI
ff11000000000000	-59.75 PB	ff90ffffffffffff	32 PB	direct mapping of all physical memory (page_offset_base)
ff91000000000000	-27.75 PB	ff9fffffffffffff	3.75 PB	unused hole
ffa0000000000000	-24 PB	ffd1ffffffffffff	12.5 PB	vmalloc/ioremap space (vmalloc_base)
ffd2000000000000	-11.5 PB	ffd3ffffffffffff	0.5 PB	unused hole
ffd4000000000000	-11 PB	ffd5ffffffffffff	0.5 PB	vmemmap (struct page array)
ffd6000000000000	-10.5 PB	ffdeffffffffffff	2.25 PB	unused hole
ffdf000000000000	-8.25 PB	fffffbffffffffff	~8 PB	KASAN shadow memory
Identical layout to the 47LA (4-level paging) from here on				
ffffffc000000000	-4 TB	fffffdffffffffff	2 TB	unused hole (vaddr_end for KASLR)
fffffe0000000000	-2 TB	fffffe7fffffffff	0.5 TB	cpu_entry_area mapping
fffffe8000000000	-1.5 TB	fffffeffffffffff	0.5 TB	unused hole
ffffff0000000000	-1 TB	ffffff7fffffffff	0.5 TB	%esp fixup stacks
ffffff8000000000	-512 GB	ffffffeeffffffff	444 GB	unused hole
ffffffef00000000	-68 GB	fffffffeffffffff	64 GB	EFI region mapping space

ffffffff00000000	-4 GB	ffffffff7fffffff	2 GB	unused hole
ffffffff80000000	-2 GB	ffffffff9fffffff	512 MB	kernel text mapping (phys=0)
ffffffffa0000000	-1536 MB	ffffffffefffffff	1520 MB	module mapping space
fffffffffff000000	-16 MB	fffffffffff5fffff	∼0.5 MB	kernel-internal fixmap range
fffffffffff600000	-10 MB	fffffffffff600fff	4 KB	legacy vsyscall ABI
fffffffffffe00000	-2 MB	ffffffffffffffff	2 MB	unused hole

Appendix

MacOS Virtual Memory Regions

This layout provides a structured overview of how memory is allocated and protected within the process address space of the MacOS Virtual Memory Regions [4]

Region	Address / Range	Size	Permissions	Sharing	Notes
==== Non-writable regions for process xxx					
__PAGEZERO	0	4K	—/—	NUL	Clock
__TEXT	1000	40K	r-x/rwx	COW	Clock
__LINKEDIT	e000	4K	r–/rwx	COW	Clock
—	90000	4K	r–/r–	SHM	—
—	340000	3228K	r–/rwx	COW	—
—	789000	3228K	r–/rwx	COW	—
Submap	90000000–9fffffff	—	r–/r–	—	Machine-wide submap
__TEXT	90000000	932K	r-x/r-x	COW	libSystem.B.dylib
__LINKEDIT	900e9000	260K	r–/r–	COW	libSystem.B.dylib
__TEXT	90130000	740K	r-x/r-x	COW	CoreFoundation
__LINKEDIT	901e9000	188K	r–/r–	COW	CoreFoundation
__TEXT	90220000	2144K	r-x/r-x	COW	CarbonCore
__LINKEDIT	90438000	296K	r–/r–	COW	CarbonCore
==== Writable regions for process xxx					
__DATA	18000	4K	rw-/rwx	PRV	TextEdit
__OBJC	19000	8K	rw-/rwx	COW	TextEdit
MALLOC_OTHER	1d000	256K	rw-/rwx	PRV	—
MALLOC_USED	5d000	256K	rw-/rwx	PRV	—
—	9d000	372K	rw-/rwx	COW	—
VALLOC_USED	ff000	36K	rw-/rwx	PRV	—
Submap	a000b000–a012ffff	—	r–/r–	—	Process-only
__DATA	a0130000	28K	rw-/rw-	COW	CoreFoundation
__DATA	a0220000	20K	rw-/rw-	COW	CarbonCore
__DATA	a0490000	12K	rw-/rw-	COW	IOKit
__DATA	a0510000	36K	rw-/rw-	COW	OSServices
—	b959e000	4K	rw-/rw-	SHM	—
—	b95a0000	4K	rw-/rw-	SHM	—
—	b9630000	164K	rw-/rw-	SHM	—
—	b965a000	896K	rw-/rw-	SHM	—
—	bff80000	504K	rw-/rwx	ZER	—
STACK[0]	bfffe000	4K	rw-/rwx	PRV	—
STACK[1]	f0001000	512K	rw-/rwx	PRV	—
—	ff002000	12272K	rw-/rw-	SHM	—

[4] https://developer.apple.com/library/archive/documentation/Performance/Conceptual/ManagingMemory/Articles/VMPages.html

Appendix

NTKernel Address Layout

Start	**End**	**Size**	**Description**
FFFF0800`00000000	FFFFF67F`FFFFFFFF	238TB	Unused System Space
FFFFF680`00000000	FFFFF6FF`FFFFFFFF	512GB	PTE Space
FFFFF700`00000000	FFFFF77F`FFFFFFFF	512GB	HyperSpace
FFFFF780`00000000	FFFFF780`00000FFF	4K	Shared System Page
FFFFF780`00001000	FFFFF7FF`FFFFFFFF	512GB-4K	System Cache Working Set
FFFFF800`00000000	FFFFF87F`FFFFFFFF	512GB	Initial Loader Mappings
FFFFF880`00000000	FFFFF89F`FFFFFFFF	128GB	Sys PTEs
FFFFF8a0`00000000	FFFFF8bF`FFFFFFFF	128GB	Paged Pool Area
FFFFF900`00000000	FFFFF97F`FFFFFFFF	512GB	Session Space
FFFFF980`00000000	FFFFFFa70`FFFFFFFF	1TB	Dynamic Kernel VA Space
FFFFFa80`00000000	*nt!MmNonPagedPoolStart-1	6TB Max	PFN Database
*nt!MmNonPagedPoolStart	*nt!MmNonPagedPoolEnd	512GB Max	Non-Paged Pool
FFFFFFFF`FFc00000	FFFFFFFF`FFFFFFFF	4MB	HAL and Loader Mappings

Appendix

Software prototype of the DMA engine

Listing 1: DMA software prototype

```
int _dma_copy(int bank, uint64_t src_offset,
                uint64_t dst_offset, uint64_t length) {
    DEBUG_PRINT("DMA CALL: bank=%d, src=%llu, dst=%llu, len=%llu\n",
                bank, src_offset, dst_offset, length);

    /* Assume that BUS addressing provided through get_ptr() routine */
    void *src = get_ptr((size_t)src_offset);
    void *dst = get_ptr((size_t)dst_offset);

    DEBUG_PRINT("Before copy:\n");
    DEBUG_PRINT("  src[0] = 0x%02x, src[1] = 0x%02x\n",
           ((uint8_t*)src)[0], ((uint8_t*)src)[1]);
    DEBUG_PRINT("  dst[0] = 0x%02x, dst[1] = 0x%02x\n",
           ((uint8_t*)dst)[0], ((uint8_t*)dst)[1]);

    DEBUG_PRINT("\nCalling DMA copy directly...\n");

    if (src && dst) {
        memcpy(dst, src, (size_t)length);
    }

    DEBUG_PRINT("\nAfter copy:\n");
    DEBUG_PRINT("  src[0] = 0x%02x, src[1] = 0x%02x\n",
           ((uint8_t*)src)[0], ((uint8_t*)src)[1]);
    DEBUG_PRINT("  dst[0] = 0x%02x, dst[1] = 0x%02x\n",
           ((uint8_t*)dst)[0], ((uint8_t*)dst)[1]);

    return src && dst ? 0 : -1;

}
```